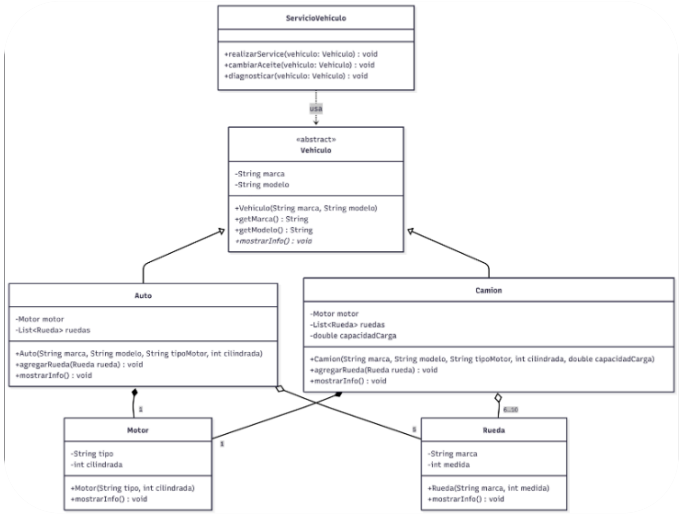
# JDBC en ejemplo vehículos




vamos a explicar paso a paso que es JDBC y como se usa aplicado al ejemplo de vehiculos: las clases eran Auto, Camion, Motor, Rueda, Vehiculo y ServicioVehiculo. Adjunto modelo de clases

Perfecto. Para este modelo, **JDBC** nos permitiría guardar y recuperar los objetos `Vehiculo`, `Auto`, `Camion`, `Motor` y `Rueda` desde una base de datos relacional.

La idea general sería:

```
Java / Objetos  →  JDBC  →  Base de datos
Auto            →         tabla auto
Camion          →         tabla camion
Motor           →         tabla motor
Rueda           →         tabla rueda
```

## 1. ¿Qué es JDBC?

**JDBC** significa **Java DataBase Connectivity**.

Es una API de Java que permite que una aplicación se conecte a una base de datos, por ejemplo:

```
Java → JDBC → MySQL
Java → JDBC → PostgreSQL
Java → JDBC → SQLite
Java → JDBC → Oracle
```

Con JDBC podemos hacer operaciones como:

```sql
INSERT
SELECT
UPDATE
DELETE
```

Es decir, guardar, consultar, modificar y eliminar datos.

---

## 2. ¿Dónde entra JDBC en nuestro modelo?

Tu modelo actual tiene clases de dominio:

```java
Vehiculo
Auto
Camion
Motor
Rueda
ServicioVehiculo
```

Estas clases representan el **modelo de objetos**.

Pero si cerramos el programa, los objetos se pierden. Por ejemplo:

```java
Auto auto = new Auto("Toyota", "Corolla", "Nafta", 1800);
```

Ese objeto existe solo en memoria.

Con JDBC, podríamos guardarlo en una base de datos:

```sql
INSERT INTO vehiculo (marca, modelo, tipo)
VALUES ('Toyota', 'Corolla', 'AUTO');
```

---

## 3. Separación recomendada

No conviene meter código JDBC directamente dentro de `Auto`, `Camion`, `Motor` o `Rueda`.

Lo correcto es separar responsabilidades:

```
modelo
 ├── Vehiculo
 ├── Auto
 ├── Camion
 ├── Motor
 └── Rueda

dao
 ├── ConexionBD
 ├── VehiculoDAO
 ├── AutoDAO
 ├── CamionDAO
 ├── MotorDAO
 └── RuedaDAO
```

Las clases del modelo representan objetos.

Las clases DAO se encargan de hablar con la base de datos.

---

## 4. ¿Qué es DAO?

**DAO** significa **Data Access Object**.

Es una clase cuya responsabilidad es acceder a la base de datos.

Por ejemplo:

Java

```java
public class AutoDAO {

    public void guardar(Auto auto) {
        // Código JDBC para insertar el auto en la BD
    }

    public Auto buscarPorId(int id) {
        // Código JDBC para buscar un auto
        return null;
    }
}
```

Entonces el `Auto` sigue siendo una clase limpia, sin SQL adentro.

---

## 5. Posible estructura de tablas

Como `Auto` y `Camion` heredan de `Vehiculo`, podemos usar una tabla principal:

```sql
CREATE TABLE vehiculo (
    id INT PRIMARY KEY AUTO_INCREMENT,
    marca VARCHAR(50),
    modelo VARCHAR(50),
    tipo VARCHAR(20)
);
```

Donde `tipo` puede ser:

```
AUTO
CAMION
```

Luego tablas específicas:

```sql
CREATE TABLE auto (
    id INT PRIMARY KEY,
    motor_id INT,
    FOREIGN KEY (id) REFERENCES vehiculo(id),
    FOREIGN KEY (motor_id) REFERENCES motor(id)
);
```

```sql
CREATE TABLE camion (
    id INT PRIMARY KEY,
    motor_id INT,
    capacidad_carga DOUBLE,
    FOREIGN KEY (id) REFERENCES vehiculo(id),
    FOREIGN KEY (motor_id) REFERENCES motor(id)
);
```

Y las clases asociadas:

```sql
CREATE TABLE motor (
    id INT PRIMARY KEY AUTO_INCREMENT,
    tipo VARCHAR(50),
    cilindrada INT
);
```

```sql
CREATE TABLE rueda (
    id INT PRIMARY KEY AUTO_INCREMENT,
    marca VARCHAR(50),
    medida INT,
```

```
    vehiculo_id INT,
    FOREIGN KEY (vehiculo_id) REFERENCES vehiculo(id)
);
```

---

## 6. Relación con el diagrama UML

En el diagrama:

```
Auto hereda de Vehiculo
Camion hereda de Vehiculo
Auto tiene Motor
Camion tiene Motor
Auto tiene varias Ruedas
Camion tiene varias Ruedas
ServicioVehiculo usa Vehiculo
```

En base de datos eso se transforma en:

```
vehiculo
 ├── auto
 └── camion

vehiculo 1 ─── 1 motor
vehiculo 1 ─── muchas ruedas
```

---

## 7. Ejemplo de conexión JDBC

Primero tendríamos una clase para conectarnos:

‹/› Java

```java
package com.universitas.dao;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConexionBD {

    private static final String URL = "jdbc:mysql://localhost:3306/vehiculos_db";
    private static final String USUARIO = "root";
    private static final String PASSWORD = "";

    public static Connection obtenerConexion() throws SQLException {
        return DriverManager.getConnection(URL, USUARIO, PASSWORD);
```

```
    }
}
```

Esta clase centraliza la conexión.

---

## 8. Ejemplo DAO para guardar un Motor

Java

```java
package com.universitas.dao;

import com.universitas.model.Motor;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class MotorDAO {

    public void guardar(Motor motor) throws SQLException {

        String sql = "INSERT INTO motor (tipo, cilindrada) VALUES (?, ?)";

        try (Connection conexion = ConexionBD.obtenerConexion();
             PreparedStatement ps = conexion.prepareStatement(sql)) {

            ps.setString(1, motor.getTipo());
            ps.setInt(2, motor.getCilindrada());

            ps.executeUpdate();
        }
    }
}
```

La línea más importante es:

Java

```java
PreparedStatement ps = conexion.prepareStatement(sql);
```

Y luego cargamos los valores:

Java

```java
ps.setString(1, motor.getTipo());
ps.setInt(2, motor.getCilindrada());
```

---

## 9. Ejemplo DAO para guardar un Vehículo

Java

```java
package com.universitas.dao;

import com.universitas.model.Vehiculo;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class VehiculoDAO {

    public void guardar(Vehiculo vehiculo) throws SQLException {

        String sql = "INSERT INTO vehiculo (marca, modelo, tipo) VALUES (?, ?, ?)";

        try (Connection conexion = ConexionBD.obtenerConexion();
             PreparedStatement ps = conexion.prepareStatement(sql)) {

            ps.setString(1, vehiculo.getMarca());
            ps.setString(2, vehiculo.getModelo());

            if (vehiculo instanceof Auto) {
                ps.setString(3, "AUTO");
            } else if (vehiculo instanceof Camion) {
                ps.setString(3, "CAMION");
            }

            ps.executeUpdate();
        }
    }
}
```

---

## 10. Papel de ServicioVehiculo

`ServicioVehiculo` no debería guardar directamente en la base de datos.

Su responsabilidad es operar sobre vehículos:

Java

```java
public class ServicioVehiculo {

    public void realizarService(Vehiculo vehiculo) {
        System.out.println("Realizando service al vehículo...");
        vehiculo.mostrarInfo();
    }
```

```java
    public void cambiarAceite(Vehiculo vehiculo) {
        System.out.println("Cambiando aceite...");
    }

    public void diagnosticar(Vehiculo vehiculo) {
        System.out.println("Diagnosticando vehículo...");
    }
}
```

Si después queremos guardar el resultado del service, podríamos agregar otra clase:

Java

```java
ServicioVehiculoDAO
```

---

## 11. Resumen conceptual para alumnos

```
Clases del modelo:
Representan objetos del problema.

DAO:
Permite guardar y recuperar esos objetos desde la base de datos.

JDBC:
Es la tecnología que usa Java para conectarse a la base de datos.

SQL:
Es el lenguaje que usamos para consultar o modificar la base.
```

---

## 12. Flujo completo

```
1. Creo un objeto Auto en Java.

2. El Auto tiene:
   - marca
   - modelo
   - motor
   - ruedas

3. Uso AutoDAO para guardarlo.

4. AutoDAO usa JDBC.

5. JDBC ejecuta sentencias SQL.
```

```
6. La información queda almacenada en la base de datos.
```

Ejemplo:

```java
Motor motor = new Motor("Nafta", 1800);

Auto auto = new Auto("Toyota", "Corolla", "Nafta", 1800);
auto.agregarRueda(new Rueda("Pirelli", 15));
auto.agregarRueda(new Rueda("Pirelli", 15));
auto.agregarRueda(new Rueda("Pirelli", 15));
auto.agregarRueda(new Rueda("Pirelli", 15));

AutoDAO autoDAO = new AutoDAO();
autoDAO.guardar(auto);
```

La idea final es que **el modelo de clases no desaparece**, sino que se complementa con una nueva capa:

```
Modelo de objetos + DAO + JDBC + Base de datos
```

la conexion la podría hacer a traves de un properties no?

Sí, y de hecho **es la forma recomendada** 👍
Usar un archivo `.properties` te permite **no hardcodear** la conexión en el código y cambiarla sin recompilar.

---

## ✅ 1. ¿Por qué usar `.properties`?

Sin properties (mala práctica):

```java
private static final String URL = "jdbc:mysql://localhost:3306/vehiculos_db";
private static final String USUARIO = "root";
private static final String PASSWORD = "";
```

Problemas:

- Si cambia la BD → tenés que recompilar
- No es portable

- No es seguro (credenciales en código)

---

## ✅ 2. Archivo `db.properties`

Lo creás dentro de:

```
src/main/resources/db.properties
```

Contenido:

properties

```properties
db.url=jdbc:mysql://localhost:3306/vehiculos_db
db.user=root
db.password=1234
db.driver=com.mysql.cj.jdbc.Driver
```

---

## ✅ 3. Clase de conexión usando properties

Java

```java
package com.universitas.dao;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.util.Properties;
import java.io.InputStream;

public class ConexionBD {

    private static String url;
    private static String user;
    private static String password;
    private static String driver;

    static {
        try (InputStream input = ConexionBD.class
                .getClassLoader()
                .getResourceAsStream("db.properties")) {

            Properties prop = new Properties();
            prop.load(input);

            url = prop.getProperty("db.url");
            user = prop.getProperty("db.user");
```

```
            password = prop.getProperty("db.password");
            driver = prop.getProperty("db.driver");

            // Cargar el driver
            Class.forName(driver);

        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public static Connection obtenerConexion() throws SQLException {
        return DriverManager.getConnection(url, user, password);
    }
}
```

---

## 🔎 4. ¿Qué está pasando acá?

### ✔️ `Properties`

Lee pares clave-valor:

```
db.url → jdbc:mysql://localhost:3306/vehiculos_db
```

---

### ✔️ `getResourceAsStream`

Busca el archivo en:

```
src/main/resources
```

Cuando hacés `mvn package`, se copia al classpath.

---

### ✔️ `static { }`

Se ejecuta **una sola vez** cuando se carga la clase.

Ideal para:

- leer config
- cargar driver
- inicializar variables

✔ `Class.forName(driver)`

Carga el driver JDBC (MySQL en este caso).

👉 En versiones modernas no siempre es necesario, pero es buena práctica didáctica.

---

## ✅ 5. Ventajas

✔ Separación de configuración
✔ No recompilás para cambiar conexión
✔ Podés tener distintos entornos:

```
db-dev.properties
db-test.properties
db-prod.properties
```

---

## ✅ 6. Mejora (muy importante para tu curso)

Podés hacerlo más profesional permitiendo elegir archivo:

```java
String entorno = System.getProperty("env", "dev");
String archivo = "db-" + entorno + ".properties";
```

Y ejecutar:

```bash
mvn exec:java -Denv=prod
```

---

## ✅ 7. Cómo encaja en tu arquitectura

```
Modelo (Auto, Camion, etc)
        ↓
DAO (AutoDAO, MotorDAO)
        ↓
ConexionBD (lee properties)
        ↓
JDBC
```

```
        ↓
Base de datos
```

---

## 💡 Para explicarlo en clase

Podés resumir así:

```
.properties = archivo de configuración

JDBC necesita:
- URL
- usuario
- contraseña

En vez de ponerlo en código:
→ lo externalizamos en properties
```

---

## 🚀 Si querés subir un nivel más

El siguiente paso natural (para tu roadmap) sería:

- usar **HikariCP (pool de conexiones)**
- o directamente **Spring Boot (DataSource automático)**

---

Si querés, en el próximo paso te armo:
👉 `AutoDAO completo (guardar Auto + Motor + Ruedas en una transacción JDBC)`
que es donde realmente se ve el poder del enfoque.

Armemos cada DAO con el CRUD, incluyamos almacenar los servicios hechos a los vehiculos

Sí. La arquitectura quedaría así:

```
model
 ├── Vehiculo
 ├── Auto
 ├── Camion
 ├── Motor
```

```
 ├── Rueda
 └── ServicioRealizado

dao
 ├── ConexionBD
 ├── MotorDAO
 ├── RuedaDAO
 ├── VehiculoDAO
 ├── AutoDAO
 ├── CamionDAO
 └── ServicioRealizadoDAO
```

## 1. Tablas SQL sugeridas

SQL

```sql
CREATE DATABASE vehiculos_db;
USE vehiculos_db;

CREATE TABLE motor (
    id INT AUTO_INCREMENT PRIMARY KEY,
    tipo VARCHAR(50) NOT NULL,
    cilindrada INT NOT NULL
);

CREATE TABLE vehiculo (
    id INT AUTO_INCREMENT PRIMARY KEY,
    marca VARCHAR(50) NOT NULL,
    modelo VARCHAR(50) NOT NULL,
    tipo_vehiculo VARCHAR(20) NOT NULL,
    motor_id INT NOT NULL,
    FOREIGN KEY (motor_id) REFERENCES motor(id)
);

CREATE TABLE auto (
    vehiculo_id INT PRIMARY KEY,
    FOREIGN KEY (vehiculo_id) REFERENCES vehiculo(id)
);

CREATE TABLE camion (
    vehiculo_id INT PRIMARY KEY,
    capacidad_carga DOUBLE NOT NULL,
    FOREIGN KEY (vehiculo_id) REFERENCES vehiculo(id)
);

CREATE TABLE rueda (
    id INT AUTO_INCREMENT PRIMARY KEY,
    marca VARCHAR(50) NOT NULL,
    medida INT NOT NULL,
    vehiculo_id INT NOT NULL,
    FOREIGN KEY (vehiculo_id) REFERENCES vehiculo(id)
```

```
);

CREATE TABLE servicio_realizado (
    id INT AUTO_INCREMENT PRIMARY KEY,
    vehiculo_id INT NOT NULL,
    tipo_servicio VARCHAR(50) NOT NULL,
    descripcion VARCHAR(255),
    fecha DATE NOT NULL,
    FOREIGN KEY (vehiculo_id) REFERENCES vehiculo(id)
);
```

---

## 2. Clase `ServicioRealizado`

Java

```
package com.universitas.model;

import java.time.LocalDate;

public class ServicioRealizado {

    private int id;
    private int vehiculoId;
    private String tipoServicio;
    private String descripcion;
    private LocalDate fecha;

    public ServicioRealizado(int vehiculoId, String tipoServicio, String descripcion, LocalDa
        this.vehiculoId = vehiculoId;
        this.tipoServicio = tipoServicio;
        this.descripcion = descripcion;
        this.fecha = fecha;
    }

    public ServicioRealizado(int id, int vehiculoId, String tipoServicio, String descripcion,
        this.id = id;
        this.vehiculoId = vehiculoId;
        this.tipoServicio = tipoServicio;
        this.descripcion = descripcion;
        this.fecha = fecha;
    }

    public int getId() {
        return id;
    }

    public int getVehiculoId() {
        return vehiculoId;
```

```java
    }

    public String getTipoServicio() {
        return tipoServicio;
    }

    public String getDescripcion() {
        return descripcion;
    }

    public LocalDate getFecha() {
        return fecha;
    }

    public void setId(int id) {
        this.id = id;
    }

    public void setVehiculoId(int vehiculoId) {
        this.vehiculoId = vehiculoId;
    }

    public void setTipoServicio(String tipoServicio) {
        this.tipoServicio = tipoServicio;
    }

    public void setDescripcion(String descripcion) {
        this.descripcion = descripcion;
    }

    public void setFecha(LocalDate fecha) {
        this.fecha = fecha;
    }
}
```

---

## 3. `MotorDAO` con CRUD

Java

```java
package com.universitas.dao;

import com.universitas.model.Motor;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class MotorDAO {
```

```
    public int guardar(Motor motor) throws SQLException {
        String sql = "INSERT INTO motor (tipo, cilindrada) VALUES (?, ?)";

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS

            ps.setString(1, motor.getTipo());
            ps.setInt(2, motor.getCilindrada());
            ps.executeUpdate();

            ResultSet rs = ps.getGeneratedKeys();

            if (rs.next()) {
                return rs.getInt(1);
            }

            throw new SQLException("No se pudo obtener el ID del motor.");
        }
    }

    public Motor buscarPorId(int id) throws SQLException {
        String sql = "SELECT * FROM motor WHERE id = ?";

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, id);
            ResultSet rs = ps.executeQuery();

            if (rs.next()) {
                return new Motor(
                        rs.getString("tipo"),
                        rs.getInt("cilindrada")
                );
            }

            return null;
        }
    }

    public List<Motor> listarTodos() throws SQLException {
        List<Motor> motores = new ArrayList<>();

        String sql = "SELECT * FROM motor";

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
```

```
                motores.add(new Motor(
                        rs.getString("tipo"),
                        rs.getInt("cilindrada")
                ));
            }
        }

        return motores;
    }

    public void actualizar(int id, Motor motor) throws SQLException {
        String sql = "UPDATE motor SET tipo = ?, cilindrada = ? WHERE id = ?";

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setString(1, motor.getTipo());
            ps.setInt(2, motor.getCilindrada());
            ps.setInt(3, id);

            ps.executeUpdate();
        }
    }

    public void eliminar(int id) throws SQLException {
        String sql = "DELETE FROM motor WHERE id = ?";

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, id);
            ps.executeUpdate();
        }
    }
}
```

---

## 4. `RuedaDAO` con CRUD

Java

```
package com.universitas.dao;

import com.universitas.model.Rueda;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;
```

```java
public class RuedaDAO {

    public void guardar(Rueda rueda, int vehiculoId) throws SQLException {
        String sql = "INSERT INTO rueda (marca, medida, vehiculo_id) VALUES (?, ?, ?)";

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setString(1, rueda.getMarca());
            ps.setInt(2, rueda.getMedida());
            ps.setInt(3, vehiculoId);

            ps.executeUpdate();
        }
    }

    public Rueda buscarPorId(int id) throws SQLException {
        String sql = "SELECT * FROM rueda WHERE id = ?";

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, id);
            ResultSet rs = ps.executeQuery();

            if (rs.next()) {
                return new Rueda(
                        rs.getString("marca"),
                        rs.getInt("medida")
                );
            }

            return null;
        }
    }

    public List<Rueda> listarPorVehiculo(int vehiculoId) throws SQLException {
        List<Rueda> ruedas = new ArrayList<>();

        String sql = "SELECT * FROM rueda WHERE vehiculo_id = ?";

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, vehiculoId);
            ResultSet rs = ps.executeQuery();

            while (rs.next()) {
                ruedas.add(new Rueda(
                        rs.getString("marca"),
                        rs.getInt("medida")
```

```
                ));
            }
        }

        return ruedas;
    }

    public void actualizar(int id, Rueda rueda) throws SQLException {
        String sql = "UPDATE rueda SET marca = ?, medida = ? WHERE id = ?";

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setString(1, rueda.getMarca());
            ps.setInt(2, rueda.getMedida());
            ps.setInt(3, id);

            ps.executeUpdate();
        }
    }

    public void eliminar(int id) throws SQLException {
        String sql = "DELETE FROM rueda WHERE id = ?";

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, id);
            ps.executeUpdate();
        }
    }
}
```

## 5. `VehiculoDAO` base

Java

```
package com.universitas.dao;

import com.universitas.model.Vehiculo;

import java.sql.*;

public class VehiculoDAO {

    public int guardarDatosBase(Vehiculo vehiculo, String tipoVehiculo, int motorId) throws S
        String sql = """
                INSERT INTO vehiculo (marca, modelo, tipo_vehiculo, motor_id)
```

```
                VALUES (?, ?, ?, ?)
                """;

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS

            ps.setString(1, vehiculo.getMarca());
            ps.setString(2, vehiculo.getModelo());
            ps.setString(3, tipoVehiculo);
            ps.setInt(4, motorId);

            ps.executeUpdate();

            ResultSet rs = ps.getGeneratedKeys();

            if (rs.next()) {
                return rs.getInt(1);
            }

            throw new SQLException("No se pudo obtener el ID del vehículo.");
        }
    }

    public void actualizarDatosBase(int id, String marca, String modelo, int motorId) throws
        String sql = """
                UPDATE vehiculo
                SET marca = ?, modelo = ?, motor_id = ?
                WHERE id = ?
                """;

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setString(1, marca);
            ps.setString(2, modelo);
            ps.setInt(3, motorId);
            ps.setInt(4, id);

            ps.executeUpdate();
        }
    }

    public void eliminar(int id) throws SQLException {
        String sql = "DELETE FROM vehiculo WHERE id = ?";

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, id);
            ps.executeUpdate();
        }
```

```
    }
}
```

---

## 6. `AutoDAO` con CRUD

Java

```java
package com.universitas.dao;

import com.universitas.model.Auto;
import com.universitas.model.Motor;
import com.universitas.model.Rueda;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class AutoDAO {

    private final MotorDAO motorDAO = new MotorDAO();
    private final RuedaDAO ruedaDAO = new RuedaDAO();
    private final VehiculoDAO vehiculoDAO = new VehiculoDAO();

    public int guardar(Auto auto) throws SQLException {
        int motorId = motorDAO.guardar(auto.getMotor());

        int vehiculoId = vehiculoDAO.guardarDatosBase(auto, "AUTO", motorId);

        String sql = "INSERT INTO auto (vehiculo_id) VALUES (?)";

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, vehiculoId);
            ps.executeUpdate();
        }

        for (Rueda rueda : auto.getRuedas()) {
            ruedaDAO.guardar(rueda, vehiculoId);
        }

        return vehiculoId;
    }

    public Auto buscarPorId(int id) throws SQLException {
        String sql = """
                SELECT v.id, v.marca, v.modelo, m.tipo, m.cilindrada
                FROM vehiculo v
```

```
                INNER JOIN auto a ON v.id = a.vehiculo_id
                INNER JOIN motor m ON v.motor_id = m.id
                WHERE v.id = ?
                """;

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, id);
            ResultSet rs = ps.executeQuery();

            if (rs.next()) {
                Auto auto = new Auto(
                        rs.getString("marca"),
                        rs.getString("modelo"),
                        rs.getString("tipo"),
                        rs.getInt("cilindrada")
                );

                List<Rueda> ruedas = ruedaDAO.listarPorVehiculo(id);

                for (Rueda rueda : ruedas) {
                    auto.agregarRueda(rueda);
                }

                return auto;
            }

            return null;
        }
    }

    public List<Auto> listarTodos() throws SQLException {
        List<Auto> autos = new ArrayList<>();

        String sql = """
                SELECT v.id
                FROM vehiculo v
                INNER JOIN auto a ON v.id = a.vehiculo_id
                """;

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                autos.add(buscarPorId(rs.getInt("id")));
            }
        }

        return autos;
```

```
    }

    public void actualizar(int id, Auto auto) throws SQLException {
        int motorId = motorDAO.guardar(auto.getMotor());

        vehiculoDAO.actualizarDatosBase(
                id,
                auto.getMarca(),
                auto.getModelo(),
                motorId
        );

        String deleteRuedas = "DELETE FROM rueda WHERE vehiculo_id = ?";

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(deleteRuedas)) {

            ps.setInt(1, id);
            ps.executeUpdate();
        }

        for (Rueda rueda : auto.getRuedas()) {
            ruedaDAO.guardar(rueda, id);
        }
    }

    public void eliminar(int id) throws SQLException {
        try (Connection con = ConexionBD.obtenerConexion()) {

            con.setAutoCommit(false);

            try {
                eliminarDependencias(con, id);
                con.commit();
            } catch (SQLException e) {
                con.rollback();
                throw e;
            }
        }
    }

    private void eliminarDependencias(Connection con, int id) throws SQLException {
        String deleteServicios = "DELETE FROM servicio_realizado WHERE vehiculo_id = ?";
        String deleteRuedas = "DELETE FROM rueda WHERE vehiculo_id = ?";
        String deleteAuto = "DELETE FROM auto WHERE vehiculo_id = ?";
        String deleteVehiculo = "DELETE FROM vehiculo WHERE id = ?";

        try (PreparedStatement ps1 = con.prepareStatement(deleteServicios);
             PreparedStatement ps2 = con.prepareStatement(deleteRuedas);
             PreparedStatement ps3 = con.prepareStatement(deleteAuto);
             PreparedStatement ps4 = con.prepareStatement(deleteVehiculo)) {
```

```
            ps1.setInt(1, id);
            ps1.executeUpdate();

            ps2.setInt(1, id);
            ps2.executeUpdate();

            ps3.setInt(1, id);
            ps3.executeUpdate();

            ps4.setInt(1, id);
            ps4.executeUpdate();
        }
    }
}
```

---

## 7. `CamionDAO` con CRUD

Java

```
package com.universitas.dao;

import com.universitas.model.Camion;
import com.universitas.model.Rueda;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class CamionDAO {

    private final MotorDAO motorDAO = new MotorDAO();
    private final RuedaDAO ruedaDAO = new RuedaDAO();
    private final VehiculoDAO vehiculoDAO = new VehiculoDAO();

    public int guardar(Camion camion) throws SQLException {
        int motorId = motorDAO.guardar(camion.getMotor());

        int vehiculoId = vehiculoDAO.guardarDatosBase(camion, "CAMION", motorId);

        String sql = "INSERT INTO camion (vehiculo_id, capacidad_carga) VALUES (?, ?)";

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, vehiculoId);
            ps.setDouble(2, camion.getCapacidadCarga());
            ps.executeUpdate();
```

```
        }

        for (Rueda rueda : camion.getRuedas()) {
            ruedaDAO.guardar(rueda, vehiculoId);
        }

        return vehiculoId;
    }

    public Camion buscarPorId(int id) throws SQLException {
        String sql = """
                SELECT v.id, v.marca, v.modelo, m.tipo, m.cilindrada, c.capacidad_carga
                FROM vehiculo v
                INNER JOIN camion c ON v.id = c.vehiculo_id
                INNER JOIN motor m ON v.motor_id = m.id
                WHERE v.id = ?
                """;

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, id);
            ResultSet rs = ps.executeQuery();

            if (rs.next()) {
                Camion camion = new Camion(
                        rs.getString("marca"),
                        rs.getString("modelo"),
                        rs.getString("tipo"),
                        rs.getInt("cilindrada"),
                        rs.getDouble("capacidad_carga")
                );

                List<Rueda> ruedas = ruedaDAO.listarPorVehiculo(id);

                for (Rueda rueda : ruedas) {
                    camion.agregarRueda(rueda);
                }

                return camion;
            }

            return null;
        }
    }

    public List<Camion> listarTodos() throws SQLException {
        List<Camion> camiones = new ArrayList<>();

        String sql = """
                SELECT v.id
```

```
                FROM vehiculo v
                INNER JOIN camion c ON v.id = c.vehiculo_id
                """;

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                camiones.add(buscarPorId(rs.getInt("id")));
            }
        }

        return camiones;
    }

    public void actualizar(int id, Camion camion) throws SQLException {
        int motorId = motorDAO.guardar(camion.getMotor());

        vehiculoDAO.actualizarDatosBase(
                id,
                camion.getMarca(),
                camion.getModelo(),
                motorId
        );

        String sql = "UPDATE camion SET capacidad_carga = ? WHERE vehiculo_id = ?";

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setDouble(1, camion.getCapacidadCarga());
            ps.setInt(2, id);
            ps.executeUpdate();
        }

        String deleteRuedas = "DELETE FROM rueda WHERE vehiculo_id = ?";

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(deleteRuedas)) {

            ps.setInt(1, id);
            ps.executeUpdate();
        }

        for (Rueda rueda : camion.getRuedas()) {
            ruedaDAO.guardar(rueda, id);
        }
    }

    public void eliminar(int id) throws SQLException {
```

```java
        try (Connection con = ConexionBD.obtenerConexion()) {

            con.setAutoCommit(false);

            try {
                eliminarDependencias(con, id);
                con.commit();
            } catch (SQLException e) {
                con.rollback();
                throw e;
            }
        }
    }

    private void eliminarDependencias(Connection con, int id) throws SQLException {
        String deleteServicios = "DELETE FROM servicio_realizado WHERE vehiculo_id = ?";
        String deleteRuedas = "DELETE FROM rueda WHERE vehiculo_id = ?";
        String deleteCamion = "DELETE FROM camion WHERE vehiculo_id = ?";
        String deleteVehiculo = "DELETE FROM vehiculo WHERE id = ?";

        try (PreparedStatement ps1 = con.prepareStatement(deleteServicios);
             PreparedStatement ps2 = con.prepareStatement(deleteRuedas);
             PreparedStatement ps3 = con.prepareStatement(deleteCamion);
             PreparedStatement ps4 = con.prepareStatement(deleteVehiculo)) {

            ps1.setInt(1, id);
            ps1.executeUpdate();

            ps2.setInt(1, id);
            ps2.executeUpdate();

            ps3.setInt(1, id);
            ps3.executeUpdate();

            ps4.setInt(1, id);
            ps4.executeUpdate();
        }
    }
}
```

---

## 8. `ServicioRealizadoDAO` con CRUD

Java

```java
package com.universitas.dao;

import com.universitas.model.ServicioRealizado;
```

```java
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class ServicioRealizadoDAO {

    public int guardar(ServicioRealizado servicio) throws SQLException {
        String sql = """
                INSERT INTO servicio_realizado
                (vehiculo_id, tipo_servicio, descripcion, fecha)
                VALUES (?, ?, ?, ?)
                """;

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS

            ps.setInt(1, servicio.getVehiculoId());
            ps.setString(2, servicio.getTipoServicio());
            ps.setString(3, servicio.getDescripcion());
            ps.setDate(4, Date.valueOf(servicio.getFecha()));

            ps.executeUpdate();

            ResultSet rs = ps.getGeneratedKeys();

            if (rs.next()) {
                return rs.getInt(1);
            }

            throw new SQLException("No se pudo obtener el ID del servicio.");
        }
    }

    public ServicioRealizado buscarPorId(int id) throws SQLException {
        String sql = "SELECT * FROM servicio_realizado WHERE id = ?";

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, id);
            ResultSet rs = ps.executeQuery();

            if (rs.next()) {
                return new ServicioRealizado(
                        rs.getInt("id"),
                        rs.getInt("vehiculo_id"),
                        rs.getString("tipo_servicio"),
                        rs.getString("descripcion"),
                        rs.getDate("fecha").toLocalDate()
                );
            }
```

```
            return null;
        }
    }

    public List<ServicioRealizado> listarPorVehiculo(int vehiculoId) throws SQLException {
        List<ServicioRealizado> servicios = new ArrayList<>();

        String sql = "SELECT * FROM servicio_realizado WHERE vehiculo_id = ?";

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, vehiculoId);
            ResultSet rs = ps.executeQuery();

            while (rs.next()) {
                servicios.add(new ServicioRealizado(
                        rs.getInt("id"),
                        rs.getInt("vehiculo_id"),
                        rs.getString("tipo_servicio"),
                        rs.getString("descripcion"),
                        rs.getDate("fecha").toLocalDate()
                ));
            }
        }

        return servicios;
    }

    public List<ServicioRealizado> listarTodos() throws SQLException {
        List<ServicioRealizado> servicios = new ArrayList<>();

        String sql = "SELECT * FROM servicio_realizado";

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                servicios.add(new ServicioRealizado(
                        rs.getInt("id"),
                        rs.getInt("vehiculo_id"),
                        rs.getString("tipo_servicio"),
                        rs.getString("descripcion"),
                        rs.getDate("fecha").toLocalDate()
                ));
            }
        }

        return servicios;
```

```java
    }

    public void actualizar(int id, ServicioRealizado servicio) throws SQLException {
        String sql = """
                UPDATE servicio_realizado
                SET tipo_servicio = ?, descripcion = ?, fecha = ?
                WHERE id = ?
                """;

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setString(1, servicio.getTipoServicio());
            ps.setString(2, servicio.getDescripcion());
            ps.setDate(3, Date.valueOf(servicio.getFecha()));
            ps.setInt(4, id);

            ps.executeUpdate();
        }
    }

    public void eliminar(int id) throws SQLException {
        String sql = "DELETE FROM servicio_realizado WHERE id = ?";

        try (Connection con = ConexionBD.obtenerConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, id);
            ps.executeUpdate();
        }
    }
}
```

---

## 9. Uso desde `ServicioVehiculo`

Ahora `ServicioVehiculo` puede operar sobre el vehículo y además registrar lo realizado:

‹/› Java

```java
package com.universitas.service;

import com.universitas.dao.ServicioRealizadoDAO;
import com.universitas.model.ServicioRealizado;
import com.universitas.model.Vehiculo;

import java.sql.SQLException;
import java.time.LocalDate;
```

```java
public class ServicioVehiculo {

    private final ServicioRealizadoDAO servicioDAO = new ServicioRealizadoDAO();

    public void realizarService(int vehiculoId, Vehiculo vehiculo) throws SQLException {
        vehiculo.mostrarInfo();

        ServicioRealizado servicio = new ServicioRealizado(
                vehiculoId,
                "SERVICE GENERAL",
                "Se realizó service general al vehículo",
                LocalDate.now()
        );

        servicioDAO.guardar(servicio);
    }

    public void cambiarAceite(int vehiculoId) throws SQLException {
        ServicioRealizado servicio = new ServicioRealizado(
                vehiculoId,
                "CAMBIO DE ACEITE",
                "Se realizó cambio de aceite",
                LocalDate.now()
        );

        servicioDAO.guardar(servicio);
    }

    public void diagnosticar(int vehiculoId) throws SQLException {
        ServicioRealizado servicio = new ServicioRealizado(
                vehiculoId,
                "DIAGNOSTICO",
                "Se realizó diagnóstico general",
                LocalDate.now()
        );

        servicioDAO.guardar(servicio);
    }
}
```

## 10. Ejemplo de uso

Java

```java
package com.universitas;

import com.universitas.dao.AutoDAO;
import com.universitas.dao.ServicioRealizadoDAO;
import com.universitas.model.Auto;
```

```java
import com.universitas.model.Rueda;
import com.universitas.model.ServicioRealizado;
import com.universitas.service.ServicioVehiculo;

import java.time.LocalDate;

public class Main {

    public static void main(String[] args) throws Exception {

        Auto auto = new Auto("Toyota", "Corolla", "Nafta", 1800);

        auto.agregarRueda(new Rueda("Pirelli", 15));
        auto.agregarRueda(new Rueda("Pirelli", 15));
        auto.agregarRueda(new Rueda("Pirelli", 15));
        auto.agregarRueda(new Rueda("Pirelli", 15));

        AutoDAO autoDAO = new AutoDAO();

        int idAuto = autoDAO.guardar(auto);

        ServicioVehiculo servicioVehiculo = new ServicioVehiculo();

        servicioVehiculo.realizarService(idAuto, auto);
        servicioVehiculo.cambiarAceite(idAuto);
        servicioVehiculo.diagnosticar(idAuto);

        ServicioRealizadoDAO servicioDAO = new ServicioRealizadoDAO();

        for (ServicioRealizado servicio : servicioDAO.listarPorVehiculo(idAuto)) {
            System.out.println(servicio.getTipoServicio() + " - " + servicio.getFecha());
        }
    }
}
```

---

Con esto ya tenés:

```
AutoDAO      → CRUD de autos
CamionDAO    → CRUD de camiones
MotorDAO     → CRUD de motores
RuedaDAO     → CRUD de ruedas
ServicioDAO  → CRUD de servicios realizados
```

La idea central para explicar a los alumnos es:

```
El modelo representa los objetos.
Los DAO se encargan de la persistencia.
JDBC permite ejecutar SQL desde Java.
La base de datos conserva la información aunque el programa se cierre.
```

bien y en el POM incluir JDBC ahora: <project
xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

```
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.universitas</groupId>
    <artifactId>proyecto-vehiculos</artifactId>
    <version>1.0-SNAPSHOT</version>

    <name>proyecto-vehiculos</name>

    <properties>

<maven.compiler.source>17</maven.compiler.source>

<maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-
8</project.build.sourceEncoding>
        <junit.version>5.10.2</junit.version>
    </properties>

    <dependencies>
        <!-- JUnit -->
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>${junit.version}</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>

            <!-- Compilador -->
```

```
        <plugin>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.11.0</version>
        </plugin>

        <!-- Ejecutar desde Maven -->
        <plugin>
            <groupId>org.codehaus.mojo</groupId>
            <artifactId>exec-maven-plugin</artifactId>
            <version>3.1.0</version>
            <configuration>

<mainClass>com.universitas.vehiculos.App</mainClass>
            </configuration>
        </plugin>

        <!-- TESTS -->
        <plugin>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.2.5</version>
            <configuration>
                <useModulePath>false</useModulePath>
            </configuration>
        </plugin>

        <!-- JAR ejecutable (MANIFEST.MF) -->
        <plugin>
            <artifactId>maven-jar-plugin</artifactId>
            <version>3.3.0</version>
            <configuration>
                <archive>
                    <manifest>

<mainClass>com.universitas.vehiculos.App</mainClass>
                    </manifest>
                </archive>
            </configuration>
        </plugin>

    </plugins>
</build>
```

```
</project>
```

Perfecto, estás en el momento justo para incorporar JDBC correctamente 👍

Primero, una aclaración clave para que lo puedas explicar bien en clase:

> 🔥 **JDBC NO se agrega como dependencia**
> Lo que agregás en Maven es el **driver de la base de datos** (MySQL, PostgreSQL, etc.)

---

## ✅ 1. Agregar dependencia JDBC (MySQL)

Para MySQL, agregás esto dentro de `<dependencies>`:

XML

```xml
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.3.0</version>
</dependency>
```

---

## ✅ 2. Tu POM completo actualizado

Te lo dejo listo, limpio y coherente con lo que ya venías trabajando:

XML

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.universitas</groupId>
    <artifactId>proyecto-vehiculos</artifactId>
    <version>1.0-SNAPSHOT</version>

    <name>proyecto-vehiculos</name>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

```
        <junit.version>5.10.2</junit.version>
    </properties>

    <dependencies>

        <!-- JUnit 5 -->
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>${junit.version}</version>
            <scope>test</scope>
        </dependency>

        <!-- DRIVER JDBC (MySQL) -->
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <version>8.3.0</version>
        </dependency>

    </dependencies>

    <build>
        <plugins>

            <!-- COMPILADOR -->
            <plugin>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.11.0</version>
            </plugin>

            <!-- EJECUCIÓN -->
            <plugin>
                <groupId>org.codehaus.mojo</groupId>
                <artifactId>exec-maven-plugin</artifactId>
                <version>3.1.0</version>
                <configuration>
                    <mainClass>com.universitas.vehiculos.App</mainClass>
                </configuration>
            </plugin>

            <!-- TESTS -->
            <plugin>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.2.5</version>
                <configuration>
                    <useModulePath>false</useModulePath>
                </configuration>
            </plugin>

            <!-- JAR EJECUTABLE -->
```

```xml
            <plugin>
                <artifactId>maven-jar-plugin</artifactId>
                <version>3.3.0</version>
                <configuration>
                    <archive>
                        <manifest>
                            <mainClass>com.universitas.vehiculos.App</mainClass>
                        </manifest>
                    </archive>
                </configuration>
            </plugin>

        </plugins>
    </build>

</project>
```

---

## ⚠️ 3. Problema clásico (muy importante para tus alumnos)

Con esto:

```bash
mvn clean package
java -jar target/proyecto-vehiculos.jar
```

### 👉 NO VA A FUNCIONAR 😅

Error típico:

```
No suitable driver found
```

¿Por qué?

👉 Porque el JAR NO incluye dependencias.

---

## ✅ 4. Solución: JAR con dependencias (fat jar)

Agregá este plugin:

```xml
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-shade-plugin</artifactId>
    <version>3.5.0</version>
```

```
    <executions>
        <execution>
            <phase>package</phase>
            <goals>
                <goal>shade</goal>
            </goals>
        </execution>
    </executions>
</plugin>
```

---

## 🚀 5. Cómo ejecutar

```bash
mvn clean package
```

Se genera:

```
target/proyecto-vehiculos-1.0-SNAPSHOT.jar
```

Ese JAR ahora:

- ✔ incluye MySQL driver
- ✔ incluye todo
- ✔ se puede ejecutar standalone

```bash
java -jar target/proyecto-vehiculos-1.0-SNAPSHOT.jar
```

---

## 🧠 6. Explicación didáctica clave

Para tus clases:

```
JDBC es parte de Java
↓
Pero necesita un driver
↓
El driver se agrega con Maven
↓
Ese driver permite conectarse a la BD
```

---

## 🎯 7. Opcional (muy bueno para avanzar después)

Podés mostrar variantes:

### PostgreSQL

```xml
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <version>42.7.3</version>
</dependency>
```

---

## 💡 8. Resumen corto tipo pizarrón

```
JDBC → API de Java
Driver → lo agrega Maven
DAO → usa JDBC
.properties → config BD
BD → guarda datos
```

---

Si querés, siguiente paso ideal para tu curso:

👉 hacer un **test JUnit que levante la conexión y pruebe DAO contra una BD real (o H2 embebida)**
👉 o armar **transacciones completas (guardar Auto + Motor + Ruedas en un solo commit)**

Ahí ya estás en nivel profesional ✌️